

IOT_SMART_HOME

Smart Home IoT Monitoring and Control System

Clean Project Summary Report

Summary Focus

This document explains what the project does, how its modules work together, what data is processed, and what the user can see in the desktop and web interfaces.

Area	Project Implementation
Communication	MQTT over the HiveMQ public broker
Data storage	Local SQLite database
Desktop interface	PyQt5 dashboard
Web interface	FastAPI with browser dashboard
Automation	JSON rule engine with alarms and relay actions
Project type	Virtual smart home IoT monitoring and control system

Contents

1. General Project Description
2. System Overview
3. System Architecture
4. Device Emulators
5. MQTT Communication
6. Data Manager
7. Rule Engine and Alarm Logic
8. Local Database
9. GUI Dashboard
10. REST API and Web Dashboard
11. Analytics and Assistant BOT
12. User-Facing Behavior
13. Project Value and Conclusion
14. Demo Plan
15. Bibliography

1. General Project Description

IOT_SMART_HOME is a virtual smart home system that demonstrates how IoT devices can communicate, report measurements, trigger automation rules and present the current home status to the user. The project is based on Python and uses MQTT as the communication protocol between all components.

The system does not require physical sensors. Instead, it uses software-based emulators that behave like real smart home devices. The emulators publish messages to an MQTT broker, and the central Data Manager receives the messages, saves the information and decides whether an alarm or actuator command is required.

The final result is a complete smart home monitoring and control environment. The user can see live measurements in a desktop GUI, inspect warnings and alarms, control the relay manually and access the current status through a web dashboard.

2. System Overview

Main Idea

The project shows how raw IoT data becomes useful information: devices publish data, the Data Manager processes it, rules detect important situations, the database stores the history, and the GUI displays the current status.

The project represents a small smart home with sensors, an actuator and a central management application. It includes climate monitoring, light monitoring, door state detection, electricity and water usage simulation and manual control through a button event.

Area	Project Implementation
DHT sensor	Room temperature and humidity monitoring
Light sensor	Light-level monitoring and automation trigger
Meter sensor	Electricity and water consumption simulation
Reed sensor	Door open or closed state
Button sensor	Manual event that can toggle the relay
Relay actuator	Controllable switch used for automation and manual control

3. System Architecture

The architecture is built around MQTT publish/subscribe communication. Device emulators publish messages to topics under the base topic `iot_smart_home`. The Data Manager subscribes to all project topics and receives telemetry, events, relay status updates and manager messages.

After a message is received, the Data Manager validates the payload, extracts numeric telemetry values and stores the information in the SQLite database. Then it sends the message data to the Rule Engine. If a rule condition is matched, the system creates an event or alarm and can also send a command to the relay actuator.

- Device Emulators publish telemetry and events.
- MQTT Broker routes topics between producers and subscribers.
- Data Manager parses messages, stores records and evaluates rules.
- SQLite Database stores devices, telemetry, events, alarms and relay commands.
- User Interfaces read the database and publish relay control commands when needed.

4. Device Emulators

The device emulators are responsible for producing realistic smart home data. They simulate the behavior of IoT devices and publish JSON messages to the MQTT broker. This makes it possible to test the complete smart home workflow without external hardware.

Area	Project Implementation
DHT_SENSOR	Publishes temperature and humidity values. High values can create warning or critical alarms.
LIGHT_SENSOR	Publishes the current light level. Low light can trigger relay automation.
METER_SENSOR	Publishes electricity and water measurements to simulate utility monitoring.
REED_SENSOR	Publishes door_open true or false for a simple security scenario.
BUTTON_SENSOR	Publishes a manual event when the user presses ENTER in the terminal.
RELAY_ACTUATOR	Receives ON/OFF/toggle commands and publishes its current state.

5. MQTT Communication

MQTT is used because it fits IoT systems well. It allows devices to publish messages without knowing who receives them, and it allows applications to subscribe only to topics that interest them. In this project, the public HiveMQ broker is used for communication.

The base topic of the project is `iot_smart_home`. All sensor telemetry, button events, relay commands, relay status messages and alarms are organized under this base topic.

```
Base topic: iot_smart_home
DHT telemetry: iot_smart_home/dht/telemetry
Button event: iot_smart_home/button/event
Relay command: iot_smart_home/relay/cmd
Alarms: iot_smart_home/alarms
Manager subscription: iot_smart_home/#
```

Example payload:

```
{
  "device": "DHT_SENSOR",
  "type": "telemetry",
  "temperature": 31.5,
  "humidity": 72.0,
  "timestamp": 1710000000
}
```

6. Data Manager

The Data Manager is the central logic of the project. It connects to the MQTT broker and subscribes to the full project topic tree. Every message published by a device or interface is received by this application.

- Parses incoming MQTT JSON messages.
- Identifies the source device and message type.
- Extracts numeric fields such as temperature, humidity, light, electricity, water, door_open and relay value.

- Stores telemetry, events, alarms and relay commands in SQLite.
- Sends parsed data to the Rule Engine and performs returned actions.

7. Rule Engine and Alarm Logic

The Rule Engine gives the project configurable automation behavior. Instead of hardcoding all conditions inside the Data Manager, the project stores rules in a JSON file. This makes the system easier to understand and easier to change.

Area	Project Implementation
Rule fields	source device, metric key, operator, threshold, severity, alarm type, message and actions
Example rules	high temperature, critical temperature, high humidity, low light, door open, high electricity, high water and button press
Possible actions	create_alarm, create_event, relay_on, relay_off and relay_toggle

Example Automation

If the temperature reaches a critical level, the Rule Engine creates a critical alarm and sends a relay ON command. This demonstrates automatic response to sensor data.

Rule example:
temperature >= 35.0 -> CRITICAL alarm + relay_on

8. Local Database

The project uses a local SQLite database. SQLite is useful for this project because it does not require a separate database server and can run immediately as a local file. The database file is created under the data folder.

Area	Project Implementation
devices	Registered device names and statuses
telemetry	Numeric measurements from sensors
events	System actions and notable events
alarms	Warning and critical situations
relay_commands	Actuator command history

9. GUI Dashboard

The desktop GUI is implemented with PyQt5. It gives the user a clear view of the smart home status. The GUI contains live cards for temperature, humidity, light level, electricity, water, door state and relay state.

The GUI also includes a recent events table and a recent alarms window. This is important because the user does not only see raw values, but also understands what happened in the system. Manual relay buttons publish relay commands to MQTT.

10. REST API and Web Dashboard

The project also includes a REST API using FastAPI. The API exposes current status, recent alarms, recent events, telemetry history and relay control. The web dashboard is a browser page that displays the latest values and allows relay ON/OFF actions.

The API includes endpoints for status, alarms, events, telemetry history and relay ON/OFF control. This keeps the browser dashboard independent from the desktop GUI while using the same backend information.

11. Analytics and Assistant BOT

The analytics module reads the SQLite database and can print a summary of collected data. It can also create charts for telemetry values such as temperature, humidity, light, electricity and water. This makes the stored data useful for analysis, not only for live monitoring.

The assistant bot is a simple text-based interface. It can answer commands such as home status, room temperature and alarms. It can also send relay on and relay off commands.

12. User-Facing Behavior

From the user point of view, the project behaves like a small smart home monitoring system. Sensors continuously generate data. The Data Manager receives and processes this data in the background. The GUI shows the current state, while the alarms area shows important warnings.

- High temperature creates a warning or critical alarm.
- Critical temperature can turn the relay ON automatically.
- Low light can activate relay automation.
- A button press can toggle the relay state.
- Door opening can create a security-related event or alarm.

13. Project Value and Conclusion

The main value of the project is that it combines several important IoT concepts into one working system. It is not only a set of separated scripts. The components communicate with each other and create a complete flow from virtual devices to user interface.

The project is modular and expandable. New sensors can be added by publishing messages to new topics. New rules can be added in the rules JSON file. New user interfaces can read the database or use the REST API.

Final Summary

IOT_SMART_HOME is a complete virtual IoT smart home project. It includes device emulators, MQTT communication, a central Data Manager, a local database, rule-based alarms, relay automation, a desktop GUI, a REST API, a web dashboard, analytics and an assistant bot.

14. Demo Plan

- Start the Data Manager.
- Start the Relay emulator.
- Start DHT, Light, Meter and Reed emulators.
- Press the Button emulator to demonstrate manual event handling.
- Open the PyQt GUI and Web Dashboard.
- Show stored DB values, alarms and relay automation.

15. Bibliography

1. **Eclipse Paho MQTT Python Client Documentation:**
<https://eclipse.dev/paho/files/paho.mqtt.python/html/>
2. **HiveMQ MQTT Essentials:** <https://www.hivemq.com/mqtt/>
3. **FastAPI Documentation:** <https://fastapi.tiangolo.com/>
4. **PyQt5 Documentation - Riverbank Computing:**
<https://www.riverbankcomputing.com/static/Docs/PyQt5/>
5. **SQLite Documentation:** <https://www.sqlite.org/docs.html>
6. **pandas Documentation:** <https://pandas.pydata.org/docs/>
7. **Matplotlib Documentation:** <https://matplotlib.org/stable/index.html>